

Requirements Understanding Document

The project, Library Management System, is a .Net Console application that manages the operations of a community library such as Member handling, Book and BookCopy management, borrowing, lending, fine calculation, membership and reporting functionalities.

Functional Requirements

- Member Management
 - Add new members
 - View all members
 - Search for users using either/or email/phone
 - Activate and Deactivate membership
- Book Management
 - Add Books(logical entity)
 - Create copies of a Book(physical entity)
 - Search books by title and author
 - Mark copies as Damaged – shouldn't allow users to borrow
- Borrowing
 - A transaction that checks the following steps: Member exists, member status is active, membership type, and that type based borrowing limit, availability of book copy, fine validation(< 500), different copy of same book not borrowed by the member.
- Returning
 - Save the return date, calculate delayed days if any, and fine based on those delay days(Rs. 10/ day)
 - Update borrowing status
 - Make book copy available
 - Generate a fine
- Fines
 - Pay Fines
 - Block borrowing If fine > 500
 - View history
- Reports
 - Currently borrowed books
 - Borrowing History, by member
 - Most borrowed books
 - View pending fines

Follow a Layered architecture, following separation of concerns to help improve maintainability. Interfaces are used to allow service layer to communicate with abstractions rather than concrete repo implementations -- Loose coupling.

Transaction Requirement

Steps:

1. Validate member
2. Validate unpaid fines
3. Validate borrowing count
4. Validate book availability
5. Create borrowing record
6. Update book copy status
7. Commit transaction

If at any of the place an error is triggered, the transaction is rolled back to ensure data consistency.

PostgreSQL Function Requirement

Calculate_member_fine(memberId) was implemented

Returns total unpaid fine for a member: Used when validating fine based requirement for borrowing.

Database Design

Entities used are:

- MembershipType : **Id**, Name, MaxBorrowLimit, MaxBorrowDays
- Member : **Id**, Name, Number, Email, IsActive, CreatedAt -- FK(MembershipTypeId)
- BookCategory : **Id**, Name
- Book : **Id**, Title, Author -- FK(CategoryId)
- BookCopy : **Id**, Status -- FK(BookId)
- Borrowing : **Id**, BorrowDate, DueDate, ReturnDate, Status -- FK(MemberId, BookCopyId)
- Fine : **Id**, Amount, IsPaid, PaidDate -- FK(BorrowingId)

Enums :

- BookCopyStatus : Available, Borrowed, Damaged
- BorrowingStatus : Active, Returned, Overdue

Relationships

MembershipType 1....N Members

BookCategory 1....N Books

Book 1....N BookCopies

Member 1....N Borrowings

BookCopy 1....N Borrowings